

CyberAudit

GitHub Actions : CI/CD Pipeline

Pipeline d'Intégration et Déploiement Continu

Project / Projet	CyberAudit Platform
Platform / Plateforme	GitHub Actions
Pipelines	Backend CI + Frontend CI
Backend runtime	Python 3.12, Django REST Framework
Frontend runtime	Node 20, React + Vite
Deploy target	Railway (auto-deploy on push to main)
Date	June 2026 / Juin 2026
Author / Auteur	Tommy JOUHANS and James ROUSSEL

1. What is CI/CD?

1. Qu'est-ce que le CI/CD ?

CI/CD stands for **Continuous Integration / Continuous Deployment**. It is a set of automated processes that run every time code is pushed to the repository, ensuring quality and enabling automatic deployment.

CI/CD signifie **Intégration Continue / Déploiement Continu**. C'est un ensemble de processus automatisés qui s'exécutent à chaque push de code dans le dépôt, garantissant la qualité et permettant le déploiement automatique.

Key concepts / Concepts clés

Continuous Integration (CI)	(EN) Automatically test and validate every code push. Catch bugs before they reach production. (FR) Tester et valider automatiquement chaque push. Détecter les bugs avant la production.
Continuous Deployment (CD)	(EN) Automatically deploy to production when CI passes. No manual steps needed. (FR) Déployer automatiquement en production quand le CI passe. Aucune étape manuelle.
GitHub Actions	(EN) GitHub's built-in CI/CD engine. Workflows defined in YAML files under github/workflows/. (FR) Moteur CI/CD intégré à GitHub. Workflows définis en YAML sous. github/workflows/
Workflow	(EN) A YAML file describing when and what to run. Each project has its own workflow. (FR) Fichier YAML décrivant quand et quoi exécuter. Chaque projet a son propre workflow.
Runner	(EN) The virtual machine that executes the pipeline steps. CyberAudit uses ubuntu-latest.(FR) La machine virtuelle qui exécute les étapes du pipeline. CyberAudit utilise ubuntu-latest.
Job	(EN) A group of steps that run on the same runner. CyberAudit has one job: 'test'. (FR) Groupe d'étapes qui s'exécutent sur le même runner. CyberAudit a un job : 'test'.

2. Pipeline Architecture

2. Architecture du Pipeline

CyberAudit has two independent CI pipelines, one for the backend, one for the frontend. Each is triggered only when relevant files change (path filtering).

CyberAudit possède deux pipelines CI indépendants, un pour le backend, un pour le frontend. Chacun est déclenché uniquement lorsque les fichiers concernés changent (filtrage par chemin).

Pipeline	File / Fichier	Trigger / Déclencheur	Steps
Backend CI	backend-ci.yml	push/PR on cyberaudit/backend/**	checkout → python → system deps → pip install → ruff → black → pytest
Frontend CI	frontend-ci.yml	push/PR on cyberaudit/frontend/**	checkout → node → npm ci → eslint → vitest → vite build
Deploy (Railway)	Automatic (Railway hook)	push to main branch	Railway pulls latest main and rebuilds the backend service automatically

Trigger rules / Règles de déclenchement

Pipelines use path filters so that pushing frontend code does not trigger the backend pipeline and vice versa. This saves CI minutes and avoids unnecessary runs.

Les pipelines utilisent des filtres de chemin pour que pousser du code frontend ne déclenche pas le pipeline backend et vice versa. Cela économise les minutes CI et évite les exécutions inutiles.

3. Backend CI Pipeline

3. Pipeline CI Backend

File: `.github/workflows/backend-ci.yml` | **Runner:** ubuntu-latest | **Working dir:** cyberaudit/backend

Full workflow file / Fichier complet

```
name: Backend CI

on:
  push:
    paths:
      - "cyberaudit/backend/**"
      - ".github/workflows/backend-ci.yml"
  pull_request:
    paths:
      - "cyberaudit/backend/**"

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: cyberaudit/backend
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: System deps (WeasyPrint)
        run: |
          sudo apt-get update
          sudo apt-get install -y \
            libcairo2 libpango-1.0-0 libpangoft2-1.0-0 \
            libharfbuzz0b libfontconfig1 fonts-liberation

      - name: Install Python deps
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install -r requirements-dev.txt

      - name: Lint (ruff)
        run: ruff check apps/ config/ --ignore E501

      - name: Format check (black)
```

```
run: black --check apps/ config/

- name: Tests with coverage
  run: pytest --cov=apps --cov-report=term-missing --cov-fail-under=40
```

Step-by-step breakdown / Détail étape par étape

#	Step / Étape	Command / Commande	Description (EN / FR)
1	Checkout	actions/checkout@v4	(EN) Downloads the repository code onto the runner VM. (FR) Télécharge le code du dépôt sur la VM runner.
2	Setup Python 3.12	actions/setup-python@v5 python-version: 3.12 cache: pip	(EN) Installs Python 3.12 on the runner. The pip cache speeds up subsequent runs by reusing downloaded packages. (FR) Installation Python 3.12 sur le runner. Le cache pip accélère les exécutions suivantes en réutilisant les paquets téléchargés.
3	System deps (WeasyPrint)	sudo apt-get install libcairo2 libpango ...	(EN) Installs Linux system libraries required by WeasyPrint to generate PDF reports (Cairo, Pango, HarfBuzz, FontConfig). (FR) Installe les bibliothèques système Linux requises par WeasyPrint pour générer des rapports PDF (Cairo, Pango, HarfBuzz, FontConfig).
4	Install Python deps	pip install -r requirements.txt pip install -r requirements- dev.txt	(EN) Installs all production dependencies (Django, DRF, Celery, WeasyPrint...) and dev dependencies (pytest, ruff, black, coverage...). (FR) Installation toutes les dépendances de production (Django, DRF, Celery, WeasyPrint...) et de développement (pytest, ruff, black, coverage...).
5	Lint : ruff	ruff check apps/ config/ -- ignore E501	(EN) Checks for Python code style violations using ruff (fast linter). --ignore E501 skips line-too-long errors. Fails if any error is found. (FR) Vérifie les violations de style Python avec ruff (linter rapide). --ignore E501 ignore les erreurs de ligne trop longue. Échoue si une erreur est trouvée.
6	Format : black	black --check apps/ config/	(EN)--check mode: verifies formatting without modifying files. Fails if any file is not formatted according to Black's rules. Run 'black apps/' locally to auto-fix. (FR) Mode --check : vérifie le formatage sans modifier les fichiers. Échoue si un fichier n'est pas formaté selon les règles de Black. Lancez 'black apps/' localement pour corriger.

7	Tests + Coverage	<pre>pytest --cov=apps --cov-report=term-missing --cov-fail-under=40</pre>	(EN) Runs all pytest tests. --cov=apps measures coverage of the apps/ directory. -cov-fail-under=40 fails the build if coverage drops below 40%. (FR) Exécute tous les tests pytest. --cov=apps mesure la couverture du dossier apps/. --cov-fail-under=40 fait échouer le build si la couverture tombe sous 40%.
---	------------------	--	--

3b. Backend Tools Explained

3b. Outils Backend Expliqués

ruff : Python Linter

ruff is an extremely fast Python linter written in Rust. It checks for common coding errors, unused imports, undefined variables, and style issues. It replaces tools like flake8, isort, and pyflakes in a single command.

ruff est un linter Python extrêmement rapide écrit en Rust. Il détecte les erreurs de code courantes, les imports inutilisés, les variables non définies et les problèmes de style. Il remplace des outils comme flake8, isort et pyflakes en une seule commande.

Command used	ruff check apps/ config/ --ignore E501
--ignore E501	Skips 'line too long' warnings (max 79 chars rule). Ignore les avertissements 'ligne trop longue' (règle max 79 chars).
Exit code	0 = no errors (CI passes). Non-zero = errors found (CI fails). 0 = aucune erreur (CI passe). Non-zéro = erreurs trouvées (CI échoue).
Fix locally	Run: ruff check apps/ --fix Lancer : ruff check apps/ --fix

black : Python Formatter

black is an opinionated Python code formatter. It reformats code to a single consistent style automatically. In CI, --check mode is used to verify formatting without changing files.

black est un formateur de code Python opinioné. Il reformate automatiquement le code vers un style cohérent. En CI, le mode --check est utilisé pour vérifier le formatage sans modifier les fichiers.

Command used	black --check apps/ config/
--check flag	(EN) Read-only mode: reports but does not modify files. (FR) Mode lecture seule : signale mais ne modifie pas les fichiers.
Exit code	(EN) 0 = already formatted (CI passes). Non-zero = needs formatting (CI fails). 0 = déjà formaté (CI passe). (FR) Non-zéro = nécessite formatage (CI échoue).
Fix locally	(EN) Run: black apps/ config/ (FR) Lancer: black apps/ config/

pytest + coverage

pytest is the Python test framework. Combined with pytest-cov, it measures code coverage, the percentage of source code executed during tests. The --cov-fail-under=40 flag enforces a minimum 40% coverage threshold.

pytest est le framework de test Python. Combiné avec pytest-cov, il mesure la couverture du code, le pourcentage de code source exécuté pendant les tests. Le flag --cov-fail-under=40 impose un seuil minimum de 40% de couverture.

Command used	pytest --cov=apps --cov-report=term-missing --cov-fail-under=40
--cov=apps	(EN) Measure coverage only for the apps/ directory (not third-party libs). (FR) Mesurer la couverture uniquement pour le dossier apps/ (pas les libs tierces).
--cov-report=term-missing	(EN) Print uncovered lines in terminal output. (FR) Afficher les lignes non couvertes dans la sortie terminal.
--cov-fail-under=40	(EN) Build fails if total coverage is below 40%. Prevents test debt. (FR) Le build échoue si la couverture totale est sous 40%. Préviend la dette de tests.
Current coverage	(EN) 121 backend tests, coverage > 40% (target to increase over time). (FR) 121 tests backend, couverture > 40% (objectif à augmenter avec le temps).

4. Frontend CI Pipeline

4. Pipeline CI Frontend

File: `.github/workflows/frontend-ci.yml` | **Runner:** ubuntu-latest | **Working dir:** cyberaudit/frontend

Full workflow file / Fichier complet

```
name: Frontend CI

on:
  push:
    paths:
      - "cyberaudit/frontend/**"
      - ".github/workflows/frontend-ci.yml"
  pull_request:
    paths:
      - "cyberaudit/frontend/**"

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: cyberaudit/frontend
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node 20
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: cyberaudit/frontend/package-lock.json

      - name: Install deps
        run: npm ci

      - name: Lint (eslint)
        run: npm run lint

      - name: Tests (vitest)
        run: npm run test:run

      - name: Build (vite)
        run: npm run build
```

Step-by-step breakdown / Détail étape par étape

#	Step / Étape	Command / Commande	Description (EN / FR)
1	Checkout	actions/checkout@v4	(EN) Downloads the repository code onto the runner VM. (FR) Télécharge le code du dépôt sur la VM runner.
2	Setup Node 20	actions/setup-node@v4 node-version: 20 cache: npm	(EN) Installs Node.js v20 LTS on the runner. The npm cache (keyed on package-lock.json) speeds up dependency installation. (FR) Installation Node.js v20 LTS sur le runner. Le cache npm (indexé sur package-lock.json) accélère l'installation des dépendances.
3	Install deps	npm ci	(EN) 'npm ci' installs exact versions from package-lock.json (unlike npm install which can update versions). Faster and deterministic. (FR) 'npm ci' installe les versions exactes depuis package-lock.json (contrairement à npm install qui peut mettre à jour). Plus rapide et déterministe.
4	Lint : ESLint	npm run lint	(EN) Checks JavaScript/TypeScript code for errors and style issues using ESLint. Fails if any lint error is found. (FR) Vérifie le code JavaScript/TypeScript pour les erreurs et problèmes de style avec ESLint. Échoue si une erreur de lint est trouvée.
5	Tests : Vitest	npm run test:run	(EN) Runs all frontend unit tests with Vitest (Vue/React testing framework compatible with Vite). test:run runs once (no watch mode). (FR) Exécute tous les tests unitaires frontend avec Vitest (framework de test compatible Vite). test:run s'exécute une seule fois (pas de mode watch).
6	Build : Vite	npm run build	(EN) Compiles and bundles the React app for production. Fails if any TypeScript error or import error exists. Confirms the app can actually be deployed. (FR) Compile et bundle l'app React pour la production. Échoue si une erreur TypeScript ou d'import existe. Confirme que l'app peut réellement être déployée.

5. Railway Auto-Deploy

5. Déploiement Automatique Railway

Railway is connected directly to the GitHub repository. When a push reaches the main branch (and Backend CI passes), Railway automatically pulls the latest code and redeploys the backend service without any manual action.

Railway est connecté directement au dépôt GitHub. Lorsqu'un push atteint la branche main (et que le Backend CI passe), Railway tire automatiquement le dernier code et redéploie le service backend sans aucune action manuelle.

Complete flow / Flux complet

1. Developer pushes code	git push origin main Code is sent to GitHub.
2. GitHub Actions triggers	(EN) Backend CI pipeline starts automatically on ubuntu-latest runner. Le pipeline. (FR) Backend CI démarre automatiquement sur le runner ubuntu-latest.
3. ruff lint check	(EN) Pipeline step: ruff check apps/ config/ --ignore E501. (FR) Step du pipeline: ruff check apps/ config/ --ignore E501
4. black format check	(EN) Pipeline step: black --check apps/ config/. (FR) Step du pipeline: black --check apps/ config/
5. pytest + coverage	(EN) Pipeline step: pytest --cov=apps --cov-fail-under=40. (FR) Step du pipeline: pytest --cov=apps --cov-fail-under=40
6. Railway webhook fires	(EN) GitHub notifies Railway that main was updated. (FR) GitHub notifie Railway que main a été mis à jour.
7. Railway rebuilds	(EN) Railway pulls main, runs Dockerfile/build, restarts the service. (FR) Railway tire main, exécute-le Dockerfile/build, redémarre le service.
8. Production updated	(EN) New code is live at cyberaudit-backend.up.railway.app. (FR) Nouveau code en ligne sur cyberaudit-backend.up.railway.app

What happens if CI fails? / Que se passe-t-il si le CI échoue ?

Failure cause	Result EN	Résultat FR
ruff finds an error	Pipeline stops at step 5. No deploy. GitHub shows red X on the commit.	Le pipeline s'arrête à l'étape 5. Pas de déploiement. GitHub affiche une croix rouge sur le commit.
black format fails	Pipeline stops at step 6. Fix by running 'black apps/' locally and pushing again.	Le pipeline s'arrête à l'étape 6. Corriger en exécutant 'black apps/' localement et en poussant à nouveau.
Test fails or coverage < 40%	Pipeline stops at step 7. No deploy. Must fix failing tests or add new tests.	Le pipeline s'arrête à l'étape 7. Pas de déploiement. Il faut corriger les tests échouants ou en ajouter.
All steps pass	GitHub shows green checkmark. Railway receives webhook and deploys automatically.	GitHub affiche une coche verte. Railway reçoit le webhook et déploie automatiquement.

6. Global Summary

6. Résumé Global

Tool / Outil	Type	Pipeline	Purpose EN	Rôle FR
Actions/checkout@v4	Action GitHub	Backend + Frontend	Download repo code onto runner	Télécharger le code du dépôt
Python 3.12	Runtime	Backend	Execute Django and testing tools	Exécuter Django et les outils de test
pip install	Package manager	Backend	Install all Python dependencies	Installer toutes les dépendances Python
ruff	Linter	Backend	Check Python code quality + style	Vérifier qualité + style du code Python
black	Formatter	Backend	Enforce consistent code formatting	Imposer un formatage cohérent du code
pytest + coverage	Test framework	Backend	Run tests + enforce 40% coverage min.	Exécuter tests + imposer 40% couverture min.
Node 20 + npm ci	Runtime	Frontend	Install exact JS dependencies	Installer les dépendances JS exactes
ESLint	Linter	Frontend	Check JavaScript/React code quality	Vérifier qualité du code JS/React
Vitest	Test framework	Frontend	Run React component unit tests	Exécuter les tests unitaires React
Vite build	Bundler	Frontend	Compile + bundle app for production	Compiler + bundler l'app pour la production
Railway deploy	CD / Deploy	Backend (auto)	Auto-deploy to production on push to main	Déploiement auto en production sur push main

Pipeline motto: If it passes CI, it ships. If it fails CI, it never reaches production.

Devise du pipeline : Si le CI passe, le code est livré. Si le CI échoue, il n'atteint jamais la production.